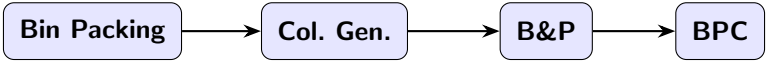
APDIO Workshop: Optimization Software & Practice

Part 3: Branch-and-Price

João Dionísio

March 27, 2026

Roadmap

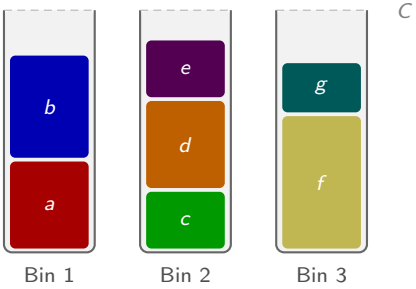


- **Column generation:** add **variables** dynamically (dual of row generation)
- **Branch-and-price:** column generation inside branch-and-bound
- **Branch-price-and-cut:** add cutting planes on top
- **Application:** Bin Packing Problem

Bin Packing Problem

Given: items with sizes s_i, bins with capacity C
Goal: pack all items using the fewest bins

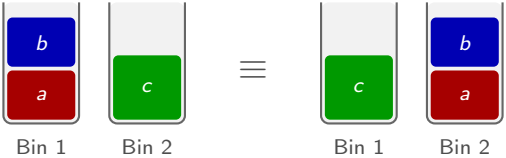
- NP-hard; applications in logistics, scheduling, cloud computing
- We will explore two formulations:
  1. Compact (assignment variables): symmetry, weak LP
  2. Extended (pattern variables): column generation



Compact Formulation

Variables: $x_{ib} \in \{0,1\}$ (item i in bin b), $y_b \in \{0,1\}$ (bin b used)

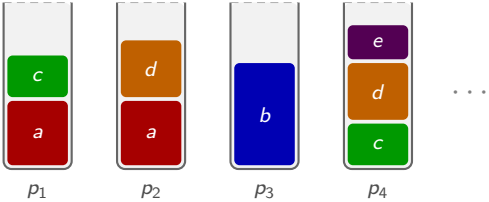
$$
\begin{aligned}
\min \quad & \sum_b y_b \\
\text{s.t.} \quad & \sum_b x_{ib} = 1 && \forall\, i \quad \text{(assign each item)} \\
& \sum_i s_i\, x_{ib} \le C\, y_b && \forall\, b \quad \text{(capacity)} \\
& x_{ib}, y_b \in \{0,1\}
\end{aligned}
$$



Same solution, different indices!

A Different Perspective: Patterns

Instead of deciding *where each item goes*, let's think about all the ways a single bin can be packed: **patterns** (or packings).



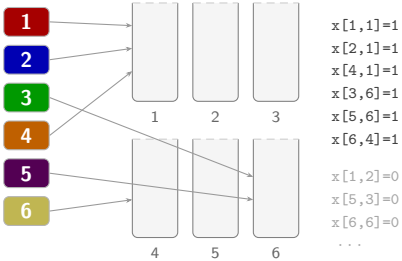
A **solution** = choose a subset of patterns that covers every item exactly once.

- No bin labels ⇒ **no symmetry**
- Problem: there are **exponentially many** patterns

Two Ways to Model a Solution

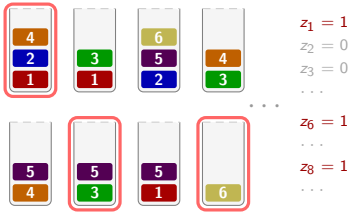
Compact

"Which bin does each item go to?"



Extended

"Enumerate packings, choose a subset"



Integer Master Problem

For all feasible patterns $\mathcal{P}$, let $a_i^p = 1$ if item i is in pattern p:

$$
\begin{aligned}
\min \quad & \sum_{p \in \mathcal{P}} z_p & & \\
\text{s.t.} \quad & \sum_{p \in \mathcal{P}} a_i^p \, z_p = 1 & & \forall\, i \in \mathcal{I} \quad (\pi_i) \\
& z_p \in \{0, 1\} & & \forall\, p \in \mathcal{P}
\end{aligned}
$$

- Exponentially many patterns $\Rightarrow$ cannot enumerate all of $\mathcal{P}$
- **Stronger LP relaxation** than compact formulation
- Solution: **Branch-and-Price!**

Approach

1. Solve the LP relaxation with **column generation**
2. Embed in branch-and-bound to get an optimal **integer** solution
3. Improve our solver!

Column Generation → **Branch-and-Price** → **Improvements**

LP Duality & Reduced Costs

Consider a general LP and its dual:

Primal

$$\begin{aligned} \min \quad & c^\top x \\ \text{s.t.} \quad & Ax = b \\ & x \geq 0 \end{aligned}$$

Dual

$$\begin{aligned} \max \quad & b^\top \pi \\ \text{s.t.} \quad & A^\top \pi \leq c \end{aligned}$$

The **reduced cost** of variable j measures its "attractiveness":

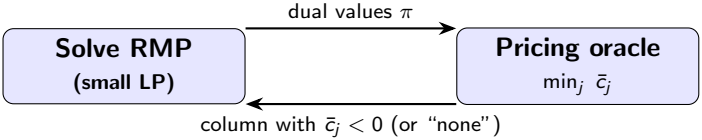
$$\bar{c}_j \;=\; c_j \;-\; \pi^\top a_j$$

- $\bar{c}_j < 0$: adding x_j to the basis can **improve** the objective
- $\bar{c}_j \geq 0 \;\; \forall j$: current solution is **LP optimal** (complementary slackness)

Simplex method: at each iteration, pivot on a variable with $\bar{c}_j < 0$ (e.g., Dantzig's rule). When none exists $\Rightarrow$ optimality certificate.

Why Column Generation Works

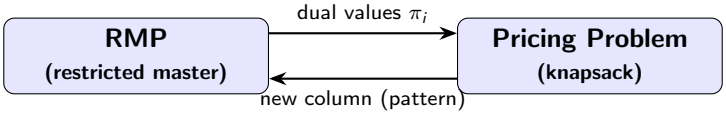
Key insight: to certify LP optimality, we only need to verify $\bar{c}_j \geq 0$ for *all* variables — but we don't need them all in the LP!



- If the pricing oracle returns $\min_j \bar{c}_j \geq 0$: the RMP solution is optimal for the **full** LP
- If $\min_j \bar{c}_j < 0$: add the column and re-solve
- We never need to enumerate all columns — just a **smart search**

Dantzig–Wolfe: exploit problem structure so that the pricing oracle is a *tractable* optimization problem (e.g., knapsack, shortest path).

Column Generation



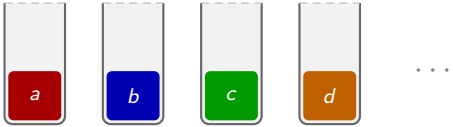
1. Solve RMP (LP relaxation with a small subset $\mathcal{P}' \subseteq \mathcal{P}$)
2. Get dual values π_i for each covering constraint
3. Solve pricing problem: find pattern with most negative reduced cost
4. If reduced cost ≥ 0: LP optimal. Else: add pattern to $\mathcal{P}'$, go to 1.

Restricted Master Problem

The RMP uses only a **subset** $\mathcal{P}' \subseteq \mathcal{P}$ of patterns:

$$
\begin{aligned}
\min \quad & \sum_{p \in \mathcal{P}'} z_p & & \\
\text{s.t.} \quad & \sum_{p \in \mathcal{P}'} a_i^p \, z_p = 1 & & \forall\, i \in \mathcal{I} \quad (\pi_i) \\
& 0 \leq z_p \leq 1 & & \forall\, p \in \mathcal{P}'
\end{aligned}
$$

Initial columns: We need the RMP to be feasible from the start.
Simplest option: one item per pattern (each item alone in a bin).



One item per bin = n initial patterns

Exercise: Initial Columns

Task

Populate the RMP with initial columns using the one-item-per-bin approach: for each item, create a pattern containing only that item.

- File: `part3/initial_columns.py`
- Test: `cd part3 && python test_initial_columns.py`

Pricing Problem

Reduced cost of a new pattern a: $\bar{c} = \underbrace{1}_{\text{obj. coeff.}} - \underbrace{\sum_i a_i \pi_i}_{\text{col. coeffs.} \times \text{duals}}$

Minimize reduced cost subject to bin capacity:

$$\begin{aligned} \min \quad & 1 - \sum_i a_i \, \pi_i \\ \text{s.t.} \quad & \sum_i s_i \, a_i \leq C, \quad a_i \in \{0, 1\} \end{aligned}$$

Pricing Problem

Reduced cost of a new pattern a: $\bar{c} = \underbrace{1}_{\text{obj. coeff.}} - \underbrace{\sum_i a_i \pi_i}_{\text{col. coeffs.} \times \text{duals}}$

Minimize reduced cost subject to bin capacity:

$$\min \quad 1 - \sum_i a_i \, \pi_i$$

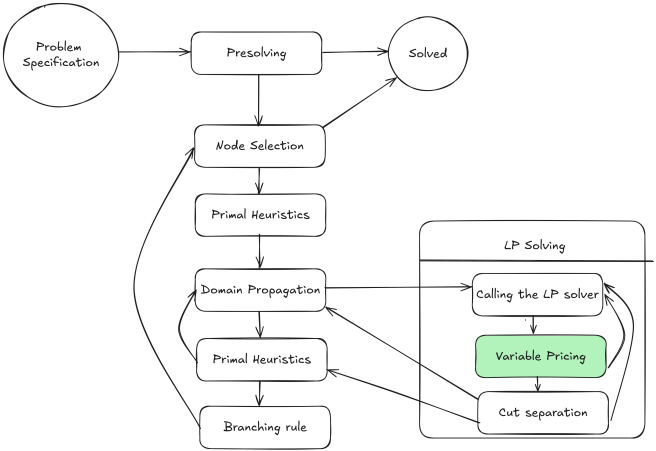
$$\text{s.t.} \quad \sum_i s_i \, a_i \leq C, \quad a_i \in \{0, 1\}$$

$= 1 - \max \sum_i a_i \pi_i \quad \Rightarrow$ **0–1 Knapsack!**

When optimal reduced cost ≥ 0: no improving pattern exists $\Rightarrow$ LP solved.

Where Does Pricing Fit in SCIP?

- After solving the LP, SCIP calls the **pricer** plugin
- The pricer adds columns with negative reduced cost
- LP is re-solved until no improving columns exist



Pricer Plugin in SCIP

Callback	Purpose
`pricerredcost`	Generate columns with negative reduced cost
`pricerfarkas`	Generate columns to restore feasibility

```
class KnapsackPricer(Pricer):
    def pricerredcost(self):
        duals = {i: self.model.getDualsolLinear(cons)
                 for i, cons in self.constraints.items()}
        red_cost, pattern = solve_knapsack(...)
        if red_cost < -1e-6:
            new_var = self.model.addVar(
                vtype="B", obj=1, pricedVar=True)
            # Add coefficients to covering constraints
        return {"result": SCIP_RESULT.SUCCESS}
```

Exercise: Knapsack Pricing

Task

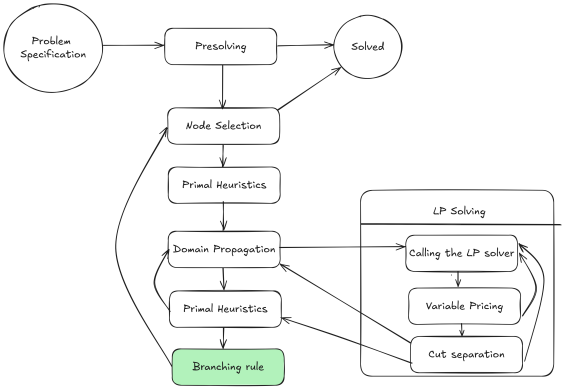
Implement the pricing subproblem. You already wrote a knapsack solver in Part 1. Copy it over and adapt it to return `(reduced_cost, pattern)`.

- File: `part3/pricing_knapsack.py`
- Test: `cd part3 && python test_pricing_knapsack.py`

Getting Integer Solutions

Column generation solves the **LP relaxation**. To get integer solutions, we embed it in **branch-and-bound**:

- At each node: solve LP with column generation
- If LP solution is fractional: **branch**
- But *how* should we branch?



Why Standard Branching Fails

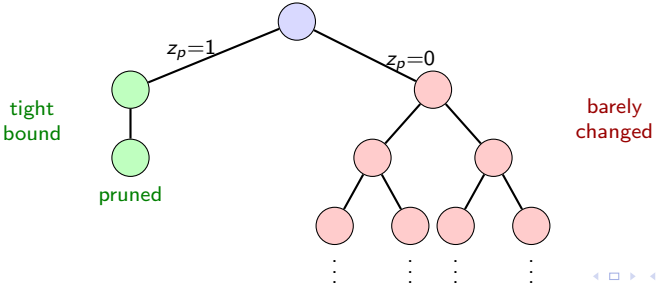
Standard variable branching on z_p:

$z_p = 1$

Force *one specific* packing out of exponentially many.
$\Rightarrow$ **Very restrictive**

$z_p = 0$

Forbid *one specific* packing out of exponentially many.
$\Rightarrow$ **Barely restrictive**



Ryan-Foster Branching

Branch on **pairs of items** (i, j) instead of individual variables:

Together

Items i and j must be in the **same** bin.

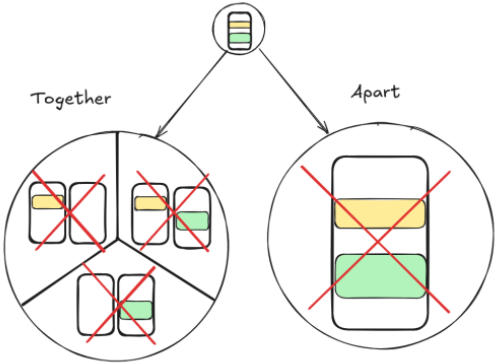
Apart

Items i and j must be in **different** bins.

How to find a pair:

- Compute implicit pair variable:
  $x_{ij} = \sum_{p:\, i \in p,\, j \in p} z_p$
- Choose pair where $0 < x_{ij} < 1$ (fractional)

$\Rightarrow$ Much more **balanced** search tree



Exercises: Ryan-Foster Branching

Exercise: Fractional Pairs

Implement `all_fractional_pairs()`: find all item pairs with fractional implicit values.

- File: `part3/ryan_foster.py`
- Test: `cd part3 && python test_fractional_pairs.py`

Exercise: Branching Decisions

Complete the child-node creation logic: inherit parent decisions, add new pair.

- File: `part3/ryan_foster.py`

Exercise: Constrained Pricing

The pricing problem must respect **all** accumulated branching decisions (**together** and **apart**).

Task

Extend the knapsack pricing to enforce together/apart constraints from Ryan-Foster branching.

- File: `part3/pricing_knapsack.py`
- Test: `cd part3 && python test_knapsack_with_constraints.py`

Integration Test

After all B&P exercises: `cd part3 && python test_bnp.py`

Compact vs. Branch-and-Price

Run the experiments

`python experiments.py` compares both formulations on increasing sizes.

Compact formulation

- Huge symmetry (bin permutations)
- Weak LP relaxation
- Scales poorly beyond $\sim$30 items

Branch-and-price

- No symmetry (patterns are unique)
- Tight LP bound (often fractional optimum $\approx$ integer)
- Scales to 100+ items

Key insight: The LP bound difference is dramatic.

Branch-Price-and-Cut

Strengthen branch-and-price by adding **cutting planes**.

Subset-row inequalities for set partitioning:
For any triple of items $S = \{i, j, k\}$, consider all packings covering ≥ 2 items in S. If their LP values sum exceeds 1:

$$\sum_{p:\, |p \cap S| \geq 2} z_p \leq 1$$

Separator plugin in SCIP:

- `sepaexeclp`: search for violated inequalities at each LP solution
- Return `SEPARATED` if cut added, `DIDNOTFIND` otherwise

Exercise: Subset-Row Separator

Task

Implement the subset-row separator: enumerate triples of fractional LP variables and add cuts when their values sum exceeds 1.

- File: `part3/subset_row.py`
- Test: `cd part3 && python test_subset_row.py`

Summary & Further Resources

What we covered:

- Bin packing: compact vs. extended formulation
- Column generation with knapsack pricing
- Branch-and-price with Ryan-Foster branching
- Branch-price-and-cut with subset-row inequalities

Bonus exercises (see `part3/README.md`):

- Dual stabilization, Lagrangian bound
- Better initialization, faster pricing
- Handling numerical issues

Resources:

- `https://www.scipopt.org/` (SCIP documentation)
- `https://github.com/scipopt/PySCIPOpt` (PySCIPOpt)